

Module 10:

Preprocessor directives, Attributes and Assemblies

Overview

- Preprocessor Directives
- Attributes and Assemblies

Preprocessor Directives

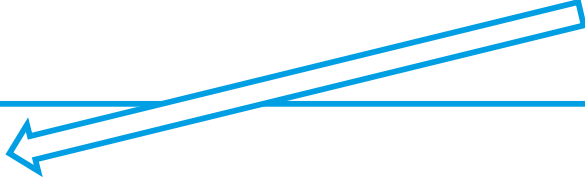
= directives to the C# compiler

- Commonly used for
 - Nullable Reference Types
 - Conditional compilation
 - Error and Warning reporting
 - Hints for Visual Editors (Region directive)
 - Line control

```
#nullable disable
    string s1 = null; // no warning
#nullable enable
    string s2 = null; // warning: possible null value
```

Conditional compilation

Conditional Compilation Symbol



```
#if TRIAL
    const int MAX_NUM_OF_USERS = 1;
#else
    const int MAX_NUM_OF_USERS = 10;
#endif
```

- Defining Conditional Compilation Symbol

```
#define TRIAL
```

```
#undef TRIAL
```

Defining Conditional Compilation Symbol

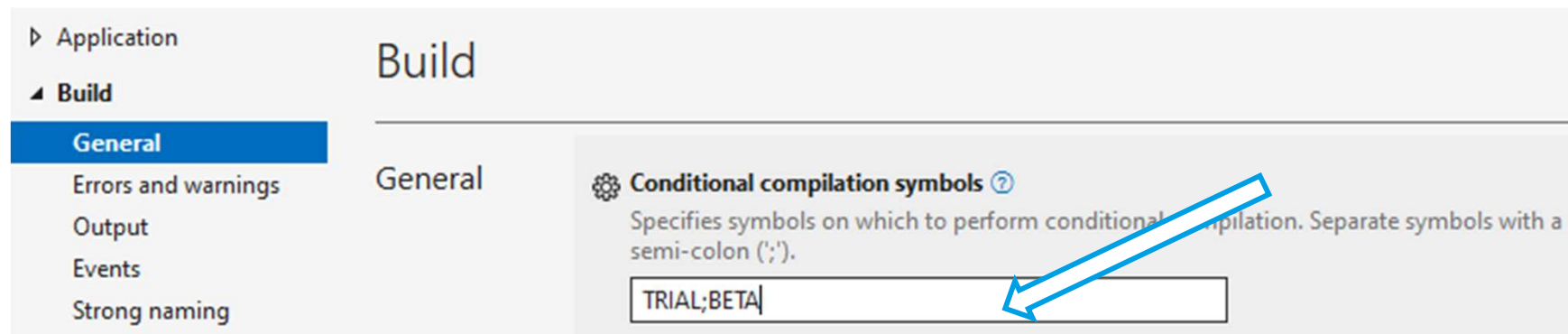
a) Define in C# code

```
#define TRIAL
```

b) Define at command line on compile time

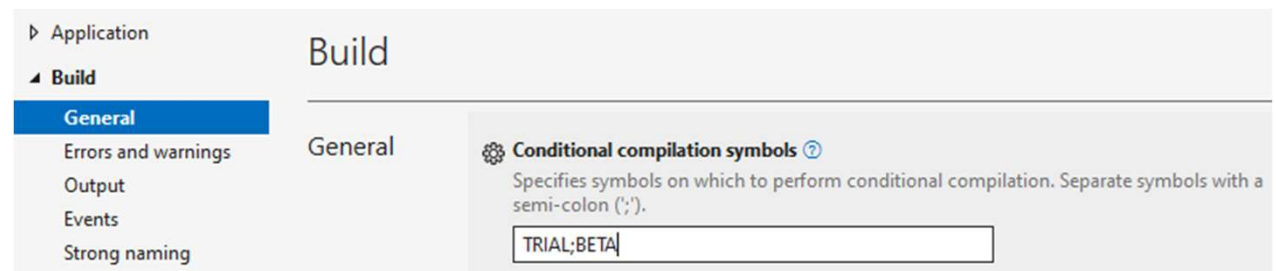
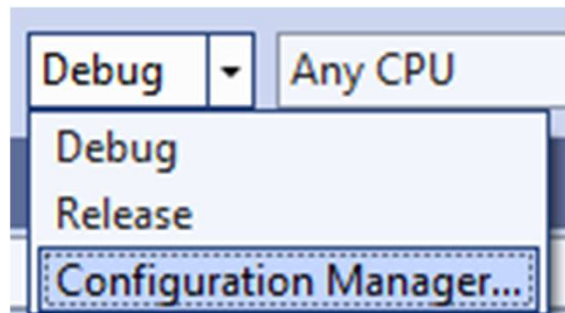
```
dotnet build -c Debug -p:DefineConstants="TRIAL"
```

c) Define in project properties

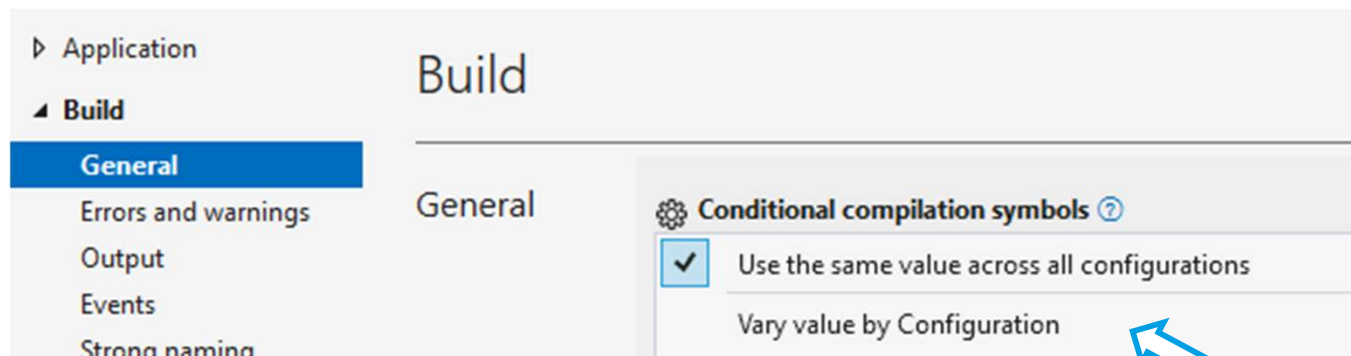


Using Configuration Manager

- Manages Release and Debug configurations for your program



- Vary values by configuration



More Conditional compilation

- Using logical operators

```
#if !DEBUG && ( BETA || PRERELEASE)

#else

#endif
```

- using #elif and #else

```
#if DEBUG

#elif RELEASE

#elif PRERELEASE

#else

#endif
```

Conditionally Executed Methods

- The compiler will not compile the method and method call if there is no compilation symbol DEBUG

```
#if DEBUG
static void WriteLog(string message)
{
    Console.WriteLine(message);
}
#endif

static void Main(string[] args)
{
    #if DEBUG
        WriteLog("Started");
    #end
}
```


Conditionally Executed Methods

- The compiler will not compile the method and method call if there is no compilation symbol DEBUG
- Method must be **void**

```
[System.Diagnostics.Conditional("DEBUG")]  
static void WriteLog(string message)  
{  
    Console.WriteLine(message);  
}  
  
static void Main(string[] args)  
{  
    WriteLog("Started");  
}
```

Region directive

- Collapsible regions

```
#region Preparing Data
    Customer obj = new Customer();
    obj.CustomerCode = "12345678901";
#endregion
```

Shortcuts:

- Ctrl+M+M - collapse/expand current region
- Ctrl+M+A - collapse all
- Ctrl+M+O - collapse all to definitions
- Ctrl+M+L - expand all

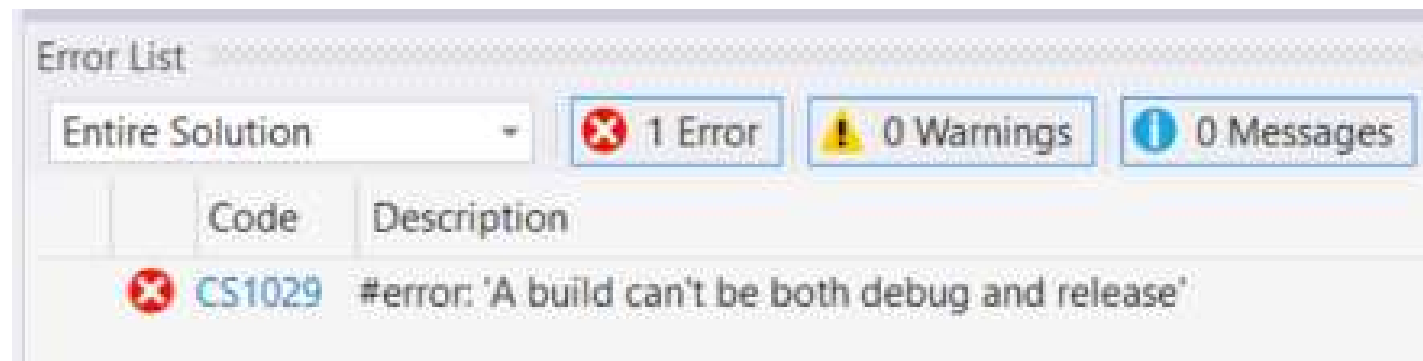
Error and Warning reporting

- Generating a warning from a specific location in your code

```
#if TRIAL  
#warning Code review needed before check-in  
#endif
```

- Generating a error from a specific location in your code

```
#if Debug && Release  
#error A build can't be both debug and release  
#endif
```



Lab 10.1 - Preprocessor Directives



Exercise:

1. Conditional compilation
2. Error and Warning reporting

Attributes

- = metadata (baked into the assembly at **compile-time**)
- Can then be accessed at runtime using reflection

- Can be used by:

- Compiler
- Visual Studio
- Addins
- Or any other tool or application

```
[System.Diagnostics.Conditional("DEBUG")]  
static void WriteLog(string message)  
{  
    Console.WriteLine(message);  
}
```

Note:

Additional information attached to your data member, property, method, class, namespace or assembly

Built-in Attributes examples

- Customizing mouse over the instance of the Type during debugging.

```
[DebuggerDisplay("FirstName={FirstName}, LastName={LastName}")]  
class Customer  
{  
    public int Id;  
    public string FirstName;  
    public string LastName;  
}
```

Built-in Attributes examples

- Marks a program entity as one that is no longer recommended for use

```
[Obsolete("This method is obsolete, use GetFullName().")]  
public string GetName()  
{  
    return this.FirstName;  
}
```

```
c.GetName();|
```

⚠ [deprecated] `string Customer.GetName()`

'Customer.GetName()' is obsolete: 'This method is obsolete, use GetFullName() instead.'

Custom Attributes

- Must Inherit from System.Attribute

```
public class AliasAttribute : Attribute
{
    public string Name { get; set; }
}
```

- Using custom Attribute

```
[Alias(Name = "Client")]
class Customer {}
```

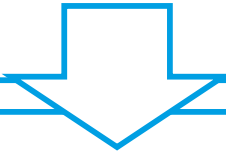

Constructors in Custom Attributes

- Creating constructor

```
public class AliasAttribute : Attribute
{
    public AliasAttribute(string name)
    {
        this.Name = name;
    }
    public string Name { get; set; }
}
```

- Using attribute constructor

```
[Alias(Name = "Client")]
class Customer {}
```



```
[Alias("Client")]
class Customer {}
```

Targeting Attributes

- This attribute can be applied only to a class

```
[AttributeUsage(AttributeTargets.Class )]
public class AliasAttribute : Attribute
{
    public string Name { get; set; }
}
```

```
[Alias(Name = "Client")]
public int Id;
```

- Combining targets

```
AttributeTargets.Class | AttributeTargets.Field
```

Using Reflection to Read Attributes

- Time of creation of the attributes is non-deterministic (they are instantiated by the CLR and you have absolutely no control over it)
- Only way how to use custom attributes is Reflection

```
[Alias(Name = "Client")]  
class Customer {}
```

```
var attributes = (AliasAttribute[])customer.GetType()  
    .GetCustomAttributes(typeof(AliasAttribute), true);  
  
foreach (var attribute in attributes)  
{  
    Console.WriteLine(attribute.Name);  
}
```



Lab 10.2 - Using Attributes



Exercise:

1. Built-In Attributes
2. Custom Attributes
3. Validation using custom attributes

Review

- Preprocessor Directives
- Attributes and Assemblies